

Lab exercises

Track 16.1 – 🐍 Python for AI Engineering · 16-ai-tech-stack/tracks/python/lab/EXERCISES.md

Lab — Python for AI Engineering

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. Solutions are intentionally not provided — the checks tell you when you're right.

Run everything with the repo venv, or your own `uv` project:

```
/Users/srinip/ai-all/.venv/bin/python 16-ai-tech-stack/tracks/python/code/<script>.py
```

1. Warm-up: read the code you were given (45 min)

Run all three scripts in `../code/`. Then:

- **1a.** In `vectorization_benchmark.py`, change `DIM` from 128 to 8 and re-run section 1. The `gemm` speedup over pure Python **drops sharply**. Explain why in one sentence. (Hint: what fraction of the work is now fixed overhead?)
- **1b.** In `async_llm_client.py`, set `fail_rate=0.0` in `demo_sequential_vs_gather`. The gathered time barely changes but the sequential time drops. Why is the *gathered* time insensitive to the failure rate?
- **1c.** In `pydantic_structured_output.py`, add `"strict": True` to `TicketAnalysis.model_config` and re-run. Exactly one previously-passing response now fails. Name it and explain what Pydantic stopped doing.

Deliverable: three sentences, one per part.

2. Set up a real project with `uv` (45 min)

From scratch, no copy-paste from the notes:

```
uv init ticket-triage && cd ticket-triage
```

Produce a project that satisfies **all** of these:

1. `src/` layout, package importable only after install.
2. `pyproject.toml` declaring `pydantic>=2.9` as a dependency and `pytest`, `ruff`, `mypy` as dev extras.
3. Ruff configured with `select = ["E", "F", "I", "B", "UP", "SIM", "ASYNC", "RUF"]`.
4. Mypy in `strict = true`.
5. A `[project.scripts]` entry point so `uv run triage --help` prints something.
6. A committed `uv.lock` and a gitignored `.venv/`.

Checks: - `uv sync && uv run ruff check . && uv run mypy src && uv run pytest` all pass on a clean clone. - `uv run python -c "import ticket_triage; print(ticket_triage.__file__)"` prints a path inside `.venv`, **not** inside `src/`. If it prints `src/`, your layout is wrong. - Delete `.venv/`, run `uv sync`, and time it. Report the number.

Deliverable: the repo, plus the `uv sync` cold-install time.

3. Make the broadcasting bug bite you (1h)

Write `centre.py` containing a function `centre(emb: np.ndarray) -> np.ndarray` that mean-centres each **column** of an `(n, d)` embedding matrix. Deliberately write the `axis=1` version.

- **3a.** Write a test with a `(4, 3)` input. It fails with a `ValueError`. Good.
- **3b.** Write a test with a `(3, 3)` input. It **passes** while computing the wrong thing. Prove it does by asserting the column means are zero — and watch the assertion fail on values that look reasonable.
- **3c.** Fix the function. Then add a shape guard that would have caught the bug regardless of squareness.

Checks: - Your guard catches the bug for `(3, 3)`, `(64, 64)` and `(1, 1)` inputs. - Your fixed version satisfies `np.allclose(centre(x).mean(axis=0), 0)` for 20 random shapes.

Deliverable: the file, plus one sentence on why `keepdims=True` would have made 3b impossible.

4. Beat the broadcast on memory (1.5h)

`dist_broadcast` in the benchmark allocates a `(300, 400, 128)` intermediate — 123 MB.

- **4a.** Scale `N_A` and `N_B` to 4,000 each and compute the intermediate size in GB before you run anything.
- **4b.** Implement `dist_chunked(A, B, chunk=512)` that produces the same answer as `dist_gemm` while never allocating more than `chunk * N_B * 8` bytes at once.
- **4c.** Measure peak memory for all three approaches with `tracemalloc` (`stdlib`) or `memray`.

Checks: - `np.allclose(dist_chunked(A, B), dist_gemm(A, B), atol=1e-8)`. - Peak allocation for `dist_chunked` is under 5% of `dist_broadcast`'s. - Your chunked version is within $2\times$ of `dist_gemm`'s wall time. If it's $10\times$ slower, your chunk is too small and you're back to paying per-call overhead.

Deliverable: a 3-row table: approach | wall time | peak MB.

5. Build the batching client for real (2h) — the centrepiece

Write `llm_batch.py`: a reusable async client wrapper. It must not hit the network — use the `FakeLLMClient` from `../code/async_llm_client.py` or your own.

Requirements:

1. A Protocol called `LLMClient` with `async def complete(self, prompt: str) -> str`.
2. `async def batch(client: LLMClient, prompts: list[str], *, max_concurrency: int, max_attempts: int) -> list[str | Exception]` — results in **input order**, failures returned not raised.
3. Bounded concurrency via `asyncio.Semaphore`.
4. Retry with exponential backoff **and full jitter**, retrying only transient errors.
5. A per-request timeout using `asyncio.timeout`.
6. A returned stats object: total HTTP calls, retries, failures, p50/p95 latency.
7. A `progress` callback invoked as each result lands (use `as_completed` or a wrapper task).

Checks: - With `fail_rate=0.3` and 200 prompts, fewer than 1% end as exceptions. - Peak in-flight requests never exceeds `max_concurrency` — assert it, don't eyeball it. - With `fail_rate=0.0`, wall time for 200 prompts at `max_concurrency=20` is within 20% of `200 / 20 * latency`. If it's much worse, you are serialising somewhere. - Setting `max_attempts=1` makes the failure count match the injected failure rate within noise. - A `BadRequestError` is raised after exactly **one** HTTP call.

Deliverable: the module, plus a printed stats table for `fail_rate ∈ {0.0, 0.1, 0.3}`.

6. A structured-output pipeline with a repair loop (2h)

Extend the Pydantic example into something you'd ship.

- **6a.** Define a `DocumentExtraction` model with at least: an enum field, a bounded numeric field, a list with `max_length`, an optional field with a default, and a nested sub-model. Put an evidence field (`supporting_quote`) **before** the judgement field it supports.
- **6b.** Write `extract(client, text, max_repairs=2)` that calls the model, validates, and on failure feeds back `ValidationError.errors()` — field paths and constraints, not `str(e)`.
- **6c.** Build a fixture file of **at least eight** bad responses: truncated JSON, fenced JSON, prose preamble, hallucinated enum, out-of-range number, wrong nesting, an extra invented field, and `null` for a required field. Assert your pipeline's behaviour on each.
- **6d.** Instrument it: count repairs, and fail the test if the repair rate over the fixture set exceeds a threshold you choose and justify.

Checks: - Every fixture produces either a valid `DocumentExtraction` or a clean, logged failure. Zero tracebacks escape `extract`. - `extra="forbid"` is set and the "invented field" fixture is rejected because of it. - Your repair prompt contains the literal field path (e.g. `citations.0.confidence`) for at least one fixture.

Deliverable: the module, the fixture file, and a passing test run in under 2 seconds.

7. Profile something and be wrong about it (1h)

Write a 60–100 line "RAG-ish" script: load ~5,000 synthetic documents, chunk them, hash them, compute fake embeddings with NumPy, and do a top-k search. Make it slow on purpose but not obviously so.

- **7a. Before profiling**, write down in a comment which function you believe dominates runtime.
- **7b.** Profile with `python -m cProfile -s cumtime`. Then with `py-spy record`.
- **7c.** Fix the actual top cost. Re-measure.
- **7d.** Compare the two profilers: what did the sampling profiler show that cProfile hid, and vice versa?

Checks: - Your 7a prediction is wrong, or your script was too obvious — rewrite it until the prediction fails. - Total speedup $\geq 3\times$ from a change to **one** function. - You can state cProfile's overhead on this script as a percentage.

Deliverable: the flame graph, the before/after numbers, and the sentence "I thought it was X, it was Y."

8. Test an LLM pipeline properly (1.5h)

Take your module from exercise 6 and give it a test suite.

1. A `FakeLLM` fixture implementing your `Protocol` — no `unittest.mock.MagicMock`.
2. A parametrised test over the eight bad fixtures from 6c.
3. A snapshot test on the **rendered prompt** (not the completion) using `syropy` or a committed golden file.
4. A seeded nondeterminism test: run the same input 10 times through a fake that returns jittered numeric values, and assert with a tolerance rather than equality.
5. A `@pytest.mark.live` test that really would call a provider, deselected by default via `addopts = "-m 'not live'"`.

Checks: - `uv run pytest` passes with no network and no API key set. Verify by unsetting every `*_API_KEY` var. - The full default suite runs in **under 5 seconds**. - Changing one word in your prompt template fails exactly one test — the snapshot test — and the diff shows the word.

Deliverable: `pytest -q` output and the snapshot diff from that one-word change.

9. The idiom hunt (45 min)

Write `bites.py`: a single file with one small, *plausible* AI-flavoured function per bug, each demonstrating the failure with a printed before/after:

1. Mutable default argument leaking conversation history between users.
2. Closure late binding producing 10 identical LLM calls instead of 10 different ones.
3. A float-equality assertion on a similarity score that fails.
4. A silent broadcasting bug (different from exercise 3 — use a mask or a bias add).
5. Unpickling untrusted data (demonstrate with a **harmless** payload, e.g. one that writes to a temp file — do not write anything destructive).
6. A bare `except:` that swallows `KeyboardInterrupt` during a long batch job.

Checks: - `ruff check bites.py` flags at least #1 (B006) and #6 (E722). Report which others it catches. - Each function prints the wrong result **and** the right one, so the diff is visible. - For #5, state in a comment what the safe alternative is for weights, for data, and for config.

Deliverable: the file plus the `ruff` output.

10. Stretch: measure the GIL (1.5h)

Prove to yourself which claims about the GIL are true.

Write one script that runs the same three workloads under `ThreadPoolExecutor(8)` and `ProcessPoolExecutor(8)` and single-threaded:

- (a) pure-Python CPU: count primes below 200,000
- (b) NumPy CPU: a large `A @ B`
- (c) simulated IO: `time.sleep(0.1) x 100`

Checks: - (a) threads give $\approx$ **no** speedup; processes give $\approx$ `n_cores`. - (b) threads give real speedup — explain why in one sentence. - (c) threads win and processes are slower than threads — explain the second half. - Print `sys._is_gil_enabled()`. If you can install a free-threaded 3.14 build, re-run (a) on it and report the delta. If you can't, say what you'd expect and why.

Deliverable: a 3×3 table of timings and three one-sentence explanations.

Exit check

Combine exercises 2, 5, 6 and 8 into one repo: `uv sync` installs it, `ruff` and `mypy` pass clean, and a sub-five-second `pytest` run exercises a rate-limited, retrying, Pydantic-validated LLM pipeline with zero network calls. Hand it to a colleague. If they get a green run before their coffee cools and can point at the line that bounds concurrency, you passed.